

ORLaTeX: write the model, not the layout

Also known as ORTeX and ORmath.

Imanol Felix Supo Mamani

Experimental version 0.1.0, September 16, 2026

1 What is ORLaTeX?

ORLaTeX is a lightweight, width-aware toolkit for mathematical optimization models. Use ordinary LaTeX mathematics; let the package arrange notation, index domains, and equation numbers. It typesets models without solving or validating them. The API is provisional; start with the lightweight workflow below. Recommended loading: `\usepackage{orlatex}`. Optional loaders: `ortex`, `ormath`; each shares the canonical implementation.

2 Quick start

Copy `orlatex.sty` and `orlatex-input.code.tex` beside your document. Save this complete example as `00-quickstart.tex` and compile twice. Here I contains n products, c_i is unit profit, and $u_i \geq 0$ is capacity.

```
\documentclass{article}
\usepackage{orlatex}
\begin{document}
\begin{ornotation}
  \set[size=n]{I}{Products}
  \param{c_i}{Unit profit}\for{i in I}
  \param{u_i}{Capacity}\for{i in I}
  \var{x_i >= 0}{Production}\for{i in I}
\end{ornotation}
\begin{ormodel}
  \maximize \sum_{i \in I} c_i x_i \\\
  \st x_i <= u_i \for{i in I} \label{capacity}
\end{ormodel}
Capacity is respected by constraint~\eqref{capacity}.
\end{document}
```

Output. Each product's production is bounded by its capacity.

$I = \{1, 2, \dots, n\}$: Products

c_i : Unit profit $\forall i \in I$

u_i : Capacity $\forall i \in I$

$x_i \geq 0$: Production $\forall i \in I$

$$\max \sum_{i \in I} c_i x_i \quad (1)$$

$$\text{s.t.} \quad x_i \leq u_i; \quad \forall i \in I \quad (2)$$

Capacity is respected by constraint (2).

3 The recommended workflow

Use `ornotation` for definitions, then `ormodel` for the objective and constraints. Short commands are local to those environments. No layout options are needed for the default workflow.

3.1 Extend the notation

Continuing the quick start, let K contain m machines. A defined set can hold pairs rather than consecutive indices; give its mathematical definition directly.

```
\begin{ornotation}
  \set[size=m]{K}{Machines}
  \setdef{A}{(i,k)\in I\times K}{Product-machine pairs}
  \param{a_{ik}}{Processing time}\for{(i,k) in A}
  \var{z_{ik}}\in\{0,1\}{Assignment}\for{(i,k) in A}
\end{ornotation}
```

$K = \{1, 2, \dots, m\}$: Machines

$A = \{(i, k) \in I \times K\}$: Product-machine pairs

a_{ik} : Processing time $\forall (i, k) \in A$

$z_{ik} \in \{0, 1\}$: Assignment $\forall (i, k) \in A$

3.2 Choose the objective sense and write rows

Minimizing negative profit gives the same optimum as maximizing profit. Use `\minimize` with a minus sign; the capacity constraint stays unchanged.

```
\begin{ormodel}
  \minimize -\sum_{i\in I} c_i x_i \\\
  \st x_i \leq u_i \for{i in I} \label{capacity-min}
\end{ormodel}
Capacity constraint:~\eqref{capacity-min}.
```

$$\min \quad - \sum_{i \in I} c_i x_i \quad (3)$$

$$\text{s.t.} \quad x_i \leq u_i; \quad \forall i \in I \quad (4)$$

Capacity constraint: (4).

3.3 The commands to remember

<code>\set</code> / <code>\setdef</code>	Name a set or give its definition.
<code>\param</code> / <code>\var</code>	Describe a parameter or complete variable declaration.
<code>\for</code>	Attach an index domain, such as <code>i in I</code> .
<code>\minimize</code> / <code>\maximize</code>	Begin the objective.
<code>\st</code>	Introduce the constraints; <code>\</code> separates rows.

Use ordinary LaTeX for mathematics. This is the complete lightweight workflow; customization and the structured API are optional.

4 Compact notation

4.1 Sets

Inside `ornotation`, use `\set[options]{symbol}{description}`. A plain set preserves your symbol without inventing members. Select at most one representation: `size`, `elements`, or `definition` (all empty by default). `\setdef{symbol}{definition}{description}` is a direct defined-set form.

```
\begin{ornotation}
  \set{C}{Customers with arbitrary identifiers}
  \set[size=n]{I}{Consecutive product indices}
  \set[elements={A,B,C}]{H}{Harvest types}
  \setdef{M}{(i,t)\in I\times T}{Product-period pairs}
\end{ornotation}
```

Here T denotes the planning-period set; define it in your application's notation. `size=|C|` explicitly requests an enumeration ending at $|C|$.

4.2 Parameters and variables

Use `\param{math}{description}` and `\var{math}{description}`. Descriptions are ordinary text and can contain inline mathematics. Group related symbols when one description is clearer. A variable is the complete declaration you supply: bounds and domains are explicit, with no implicit variable type.

```
\begin{ornotation}
  \set[size=H]{T}{Periods}
  \param{c_{it},f_{it},h_{it}}{Production, setup, and holding costs}
  \for{i in I,t in T}
  \var{x_{it} >= 0}{Production}\for{i in I,t in T}
  \var{y_{it}\in\{0,1\}}{Setup indicator}\for{i in I,t in T}
\end{ornotation}
```

4.3 Index domains with `\for`

Attach `\for{i in I,t in T}` to the preceding declaration or model row. Native membership, such as `\for{i\in I,t\in T}`, works too. Tuple memberships are supported. For arbitrary mathematical conditions, use `\for*{t>0}`; the starred form renders raw mathematics without membership parsing.

4.4 Controlled shorthand

Input	Rendered meaning
<code>x >= 0</code>	$x \geq 0$
<code>x <= U</code>	$x \leq U$
<code>\for{i in I}</code>	$\forall i \in I$
<code>\for{i in I,j in J}</code>	$\forall i \in I, j \in J$
<code>\for{(i,j) in A}</code>	$\forall (i,j) \in A$

Inequality shorthand applies at the top level in raw variables and lightweight model rows; `in` applies only inside unstarred `\for`. It is local and preserves ordinary LaTeX. Write native operators inside braced groups or macro arguments. There is no `!=` or `==` shorthand: use `\neq` and `=`. In particular, `n!=k` keeps factorial followed by equality.

5 Models

5.1 Objectives and constraints

For a minimum-cost variant, let $p_i \geq 0$ be unit cost and $d_i \geq 0$ required production. Keep $x_i \geq 0$ as in the quick start. Begin literally with `\minimize` or `\maximize`. Exactly one objective is required. Put it first, then the constraints. Use explicit `\\` between rows; blank lines are whitespace. `\st` introduces constraints; `\subjectto` is its longer synonym. `\for` supplies the index domain.

```
\begin{ormodel}
  \minimize \sum_{i \in I} p_i x_i \label{cost} \\
  \st x_i >= d_i \for{i in I} \label{minimum} \\
  x_i >= 0 \for{i in I} \notag
\end{ormodel}
```

$$\min \sum_{i \in I} p_i x_i \quad (5)$$

$$\text{s.t.} \quad x_i \geq d_i; \quad \forall i \in I \quad (6)$$

$$x_i \geq 0; \quad \forall i \in I$$

5.2 Labels and references

Use ordinary top-level `\label`, then `\ref` or `\eqref` in text. The objective is (5); the minimum-production constraint is (6). Compile twice. `\notag` suppresses one row's number and counter increment; that row cannot carry a label. Definitions in `ornotation` have no equation number and cannot carry labels.

numbering=rows Default: one number per numbered row; row labels allowed.
numbering=model One number beside the first row; use only an environment label.
numbering=none No counter increment; labels are rejected.

With row numbering, an environment label refers to the first numbered row. Do not combine `\notag` with `numbering=model`.

5.3 Long expressions and manual breaks

Protect an internally multiline expression with one outer brace group. The internal `&` and `\\` then reach amsmath unchanged; the following outer `\\` still separates model rows. These are authored breaks.

```
\begin{ormodel}
  \minimize {\&\sum_{i \in I} p_i x_i \\
             &\&{}+\sum_{k \in K} f_k y_k} \\
  \st x_i >= 0 \for{i in I}
\end{ormodel}
```

Here f_k, y_k are machine setup costs and indicators. Use `{\&+}` or `{\&-}` to preserve binary-operator spacing on a continuation line. ORLaTeX does not automatically split arbitrary mathematical expressions.

6 Responsive layout

The same source adapts to the surrounding `\linewidth`. This feasibility model has activities J , resource constraints I , periods T , resource usage a_{ij} , and capacities b_{it} . The constant objective asks only for feasibility.

```
\begin{ormodel}[numbering=none]
  \minimize 0 \\\
  \st \sum_{j\in J} a_{ij} x_{jt} <= b_{it}
  \for{i in I,t in T} \\\
  x_{jt} >= 0 \for{j in J,t in T}
\end{ormodel}
```

Wide output

```
min 0

s.t.  \sum_{j\in J} a_{ij} x_{jt} \leq b_{it}; \quad \forall i \in I, t \in T

      x_{jt} \geq 0; \quad \forall j \in J, t \in T
```

Narrow output (145pt local width)

```
min 0

s.t.  \sum_{j\in J} a_{ij} x_{jt} \leq b_{it};
      \forall i \in I, t \in T
      x_{jt} \geq 0; \quad \forall j \in J, t \in T
```

Both outputs read the identical fixture. The long capacity row needs its domain below; the shorter nonnegativity row can retain its domain inline. Neither the mathematics nor the font size changes.

6.1 What the defaults do

Notation and models begin at the local left edge. Each definition keeps its mathematical declaration, colon, description, and domain on one line when they fit. Safe gaps decrease before long descriptions wrap with hanging indentation; domains move below only when necessary. Short rows stay compact beside long rows. No automatic math-font styling, font shrinking, or margin invasion occurs. User punctuation remains literal; model rows with domains default to a semicolon.

6.2 Where width adaptation stops

The model renderer measures expressions and index domains and reserves marker and equation-number space. It adapts component placement, without inferring mathematics or scaling boxes. Long unbreakable expressions still need authored breaks. A multiline row is indivisible and cannot split across pages.

6.3 Local model domains

Every model row places its domain immediately after its own expression. Equation tags remain separate from domains and use the adaptive placement described below. With `domain-gap=auto`,

the preferred and maximum gap is 0.8em and the minimum is 0.3em, measured in the row's selected font. The renderer reserves marker and tag space, measures the expression (including any separator) and domain, and uses the remaining width to reduce the gap. If they still cannot fit, only that domain moves below. A longer neighboring row never changes the gap or placement of a short row.

Use `domain-gap=1em` or `domain-gap=10pt` for a fixed model gap. `domain-position=right` forces inline placement and can overflow; `domain-position=below` forces a continuation line. No two-column switch, font shrinking, or automatic margin extension is used. `layout=wide` retains its existing forced-inline meaning; stacked layout takes precedence.

The default is `domain-separator={;}`, following the author's editorial preference.

Select `domain-separator={,}` for commas.

Select `domain-separator={}` for none. The renderer appends punctuation only to expressions with domains, including when the domain moves below. A literal trailing comma, semicolon, or period takes precedence, avoiding doubled punctuation. Punctuation hidden in a macro or behind a spacing command is not inferred; in that case author the punctuation and leave the separator empty. The separator applies to models; notation retains its colon/prose conventions.

6.4 Compact responsive equation tags

The defaults are `tag-gap=auto` and `tag-position=auto`. Available `\linewidth` limits a model; it is not a width the model must fill. Objectives and constraints share a compact tag anchor when their natural widths permit it. A much longer row extends its own tag locally instead of pushing every short row toward the page edge. No class names or page geometry are tested.

```
\begin{ormodel}
  \minimize \sum_i c_i x_i \\
  \st x_i <= 10 \for{i in I} \\
  \sum_j a_{ij}x_j <= b_i \for{i in I} \\
  y_i+z_i <= 1 \for{i in I}
\end{ormodel}
```

$$\min \quad \sum_i c_i x_i \quad (7)$$

$$\text{s.t.} \quad x_i \leq 10; \quad \forall i \in I \quad (8)$$

$$\sum_j a_{ij} x_j \leq b_i; \quad \forall i \in I \quad (9)$$

$$y_i + z_i \leq 1; \quad \forall i \in I \quad (10)$$

For each row, the renderer measures the marker, expression, domain and actual upcoming tag in the selected font. Auto tag separation prefers one twenty-fifth of the available width after markers, bounded between 0.75em and 2em. It can compress to 0.5em when needed. Auto domain and tag gaps compress before a domain moves below. The tag then remains on the expression's first line, and the domain continuation can use the width below it. Mathematics is not shrunk.

Natural row extents determine a shared anchor with a bounded extension from the central row width. Exceptional rows keep a local tag. Shared alignment adds padding to shorter rows, but never consumes arbitrary spare page width. The implementation and outlier rule are documented in ARCHITECTURE.md.

Use `tag-gap=0.8em` for a tighter exact gap.

Use `tag-gap=2em` for more breathing room, or `tag-gap=12pt` for a dimension. An explicit gap places each tag that exact distance after its own first-line content; tags may consequently have different horizontal positions. These options apply at global setup, named-style, model and structured-row levels.

Use `tag-position=right` when a publisher requires conventional outer-right tags. This mode takes precedence over the compact anchor; `tag-gap` controls the minimum fitting reservation rather than the distance to the right edge. Forced/manual choices can overflow an extremely narrow context. The existing `numbering=rows`, `numbering=model`, `numbering=none`, ordinary labels/references and `\notag` retain their counter semantics.

7 Customization and styles

Begin with the defaults. For repeated visual choices, define a style and select it on an environment. This complete fragment uses the same lightweight rows.

```
\orlaxstyle{paper}{density=compact,row-gap=1.5pt}
\begin{ormodel}[style=paper]
  \minimize \sum_{i\in I} p_i x_i \\\
  \st x_i >= d_i \for{i in I}
\end{ormodel}
```

`\orlaxsetup{density=normal}` changes document defaults. Setup and style definitions follow TeX grouping: top-level definitions persist, grouped ones remain local. Styles contain visual keys and cannot select another **style**. The precedence is element options, environment options, named style, global setup, then defaults. Within a layer, explicit spacing overrides density, regardless of key order.

7.1 Option reference

Defaults below describe the compact preset. All environments accept visual keys, **style**, and **title**. Model-only keys are identified explicitly. Dimensions must be nonnegative TeX dimensions.

Option	Accepted values	Default	Meaning
layout	auto, wide, stacked	auto	Automatic placement or explicit component layout.
domain-position	auto, right, below	auto	Domain placement; forced right can overflow a narrow model.
font-size	auto, normal, small, footnotesize, scriptsize	auto	Inherit the font size or explicitly select one.
density	relaxed, normal, compact, dense	compact	Set row and outer spacing; automatic notation gap.
row-gap	dimension	2pt	Additional space after each row.
space-before	dimension	4pt	Vertical space before a block.
space-after	dimension	4pt	Vertical space after a block.
domain-gap	auto or dimension	auto	Bounded model gap; density-dependent notation gap.
domain-separator	math tokens	;	Model punctuation before a domain.
tag-gap	auto or dimension	auto	Model tag separation; explicit dimensions are row-local.
tag-position	auto, right	auto	Compact responsive model anchor or conventional outer-right tags.
extend-left	dimension	0pt	Explicit width into the left margin.
extend-right	dimension	0pt	Explicit width into the right margin.
style	defined style name	empty	Apply a named visual style.
title	text	empty	Add a block title.
numbering	rows, model, none	rows	Model equation-number policy.
label	label key	empty	Model environment reference.
subject-to	text	s.t.	Model constraint marker; e.g. {s.a.} .

7.2 Spacing presets and placement details

These values are reference information; ordinary writing needs no spacing setup.

Density	Row gap	Before	After	Domain gap
relaxed	7pt	12pt	12pt	16pt
normal	4pt	8pt	8pt	12pt
compact	2pt	4pt	4pt	8pt
dense	1pt	2pt	2pt	6pt

The last column describes automatic notation gaps. All presets now retain `domain-gap=auto`; models use the bounded relative gap described above. In notation, the default 8pt prose-domain gap may decrease to 3pt after a line fails to fit. The 3pt colon-prose gap may decrease to 2pt. This occurs before wrapping, without shrinking the mathematical font.

`layout=wide` Model components stay side by side; notation prose still wraps when needed.

`layout=stacked` Model domains go below their expressions; notation prose goes below its mathematical prefix.

`domain-position` In auto/wide models, right forces a side domain; stacked layout takes precedence. In notation, auto/right flow with prose when width permits; below places the domain on a continuation line after the description.

7.3 Element-level styles

Element-level control is available through the advanced structured API in the next section. Its row keys are `font-size`, `row-gap`, `domain-gap`, `domain-separator`, `domain-position`, `tag-gap`, and `tag-position`. A row style may contain only those keys. It overrides environment values, then explicit element options override it. Styles never change mathematical notation or objective sense.

8 Advanced structured API

The lightweight API is recommended for ordinary model writing. The structured API supplies additional metadata and element-level visual control, useful for advanced customization or assembled rows. Both feed the same semantic records and renderer where applicable; they are not separate layout engines.

`orsets`, `orparameters`, and `orvariables` hold structured `\set`, `\parameter`, and `\variable` declarations. Model rows use `\objective` and `\constraint`. Keep one model input style per environment. This example uses the minimum-cost variant from the models section.

```
\begin{orsets}
  \set[size=n]{I}{Products}
\end{orsets}
\begin{orparameters}
  \parameter[for={i\in I}]{p_i}{Unit cost}
  \parameter[for={i\in I}]{d_i}{Required production}
\end{orparameters}
\begin{orvariables}
  \variable[type=nonnegative,for={i\in I}]{x_i}{Production}
\end{orvariables}
\begin{ormodel}
  \objective[min]{\sum_{i\in I}p_i x_i}
  \constraint[for={i\in I},font-size=small]{x_i\geq d_i}
\end{ormodel}
```

$I = \{1, 2, \dots, n\}$: Products

p_i : Unit cost $\forall i \in I$

d_i : Required production $\forall i \in I$

$x_i \geq 0$: Production $\forall i \in I$

$$\min \sum_{i \in I} p_i x_i \quad (11)$$

$$\text{s.t.} \quad x_i \geq d_i; \quad \forall i \in I \quad (12)$$

8.1 Structured metadata reference

for	Native mathematical index domain; empty by default.
label	Model-row reference; empty by default, subject to numbering policy.
type	Variable: continuous (default), nonnegative, integer, binary.
min / max	Objective sense; min is the default.
style	Named row style, plus explicit row keys from customization.

Structured variable types append a mathematical domain. Continuous and integer variables are unrestricted in sign; add nonnegativity explicitly if needed. The inherited upright R and Z letters require no extra math font package. Raw lightweight `\var` never appends a type.

Set representation keys **size**, **elements**, and **definition** have the same meanings as in compact notation. Model objectives and constraints accept **for**, **label**, and row visual keys. For example, metadata after the sense can be written as `\objective[max,label=obj:profit]{c^T x}` inside a structured model. Labels on notation declarations are rejected.

8.2 Namespaced forms and scope

Namespaced structured commands are `\orset`, `\orparameter`, `\orvariable`, `\orobjective`, and `\orconstraint`. Lightweight adapters also expose `\orvar`, `\orsetdef`, and `\orfor` in their checked contexts. The short aliases exist only within their corresponding ORLaTeX environments; prior meanings are restored afterwards. Namespaced commands diagnose use outside the required context.

8.3 Package loading

Recommended loading remains `\usepackage{orlatex}`. The compatibility loaders are listed below; each loads the same implementation.

Recommended	<code>\usepackage{orlatex}</code>
Compatibility	<code>\usepackage{ortex}</code>
Compatibility	<code>\usepackage{ormath}</code>

If using an alias, copy its wrapper file alongside the two canonical runtime files. Independent and duplicate loading are tested. No claim is made about CTAN availability or uniqueness of the names.

9 Realistic example

This optional production-routing formulation combines four cost terms, inventory balances, conditional arc sums, shared capacity, and integer/binary decisions. It uses the advanced structured API just described. The repository also has lightweight planning examples and an API comparison using the same renderer.

The set $\mathcal{I}$ indexes sites and destinations, $\mathcal{T}$ periods, $\mathcal{M} = \mathcal{I} \times \mathcal{T}$ site-period pairs, and $\mathcal{A}$ feasible shipment arcs by period. Variables x, s, z, y represent production, inventory, shipment quantity, and setup. Parameters c, f, q, h are costs; d, U, B, a, D, S and the initial/target stock values specify demands and capacities. Full definitions are in `examples/fixtures/planning-data.tex`.

Multi-period production and routing

$$\min \sum_{(i,t) \in \mathcal{M}} c_{it} x_{it} \quad (13)$$

$$+ \sum_{(i,t) \in \mathcal{M}} f_{it} y_{it}$$

$$+ \sum_{(i,j,t) \in \mathcal{A}} q_{ijt} z_{ijt}$$

$$+ \sum_{(i,t) \in \mathcal{M}} h_{it} s_{it}$$

$$\text{s.t.} \quad x_{it} + s_{i,t-1} = d_{it} + s_{it}; \quad \forall (i, t) \in \mathcal{I} \times \mathcal{T} \quad (14)$$

$$x_{it} \leq U_{it} y_{it}; \quad \forall i \in \mathcal{I}, t \in \mathcal{T} \quad (15)$$

$$\sum_{i \in \mathcal{I}} a_i x_{it} \leq B_t; \quad \forall t \in \mathcal{T} \quad (16)$$

$$\sum_{j: (i,j,t) \in \mathcal{A}} z_{ijt} = x_{it}; \quad \forall (i, t) \in \mathcal{M} \quad (17)$$

$$\sum_{i: (i,j,t) \in \mathcal{A}} z_{ijt} \leq D_{jt}; \quad \forall (j, t) \in \mathcal{M} \quad (18)$$

$$s_{i0} = s_i^{\text{init}}; \quad \forall i \in \mathcal{I} \quad (19)$$

$$s_{iT} \geq s_i^{\text{target}}; \quad \forall i \in \mathcal{I} \quad (20)$$

$$s_{it} \leq S_i; \quad \forall (i, t) \in \mathcal{M} \quad (21)$$

$$x_{it}, s_{it} \geq 0; \quad \forall (i, t) \in \mathcal{M} \quad (22)$$

$$z_{ijt} \in \mathbf{Z}_{\geq 0}; \quad \forall (i, j, t) \in \mathcal{A} \quad (23)$$

$$y_{it} \in \{0, 1\}; \quad \forall (i, t) \in \mathcal{M} \quad (24)$$

The shared body is used in `05-long-model.tex`, `06-two-column.tex`, and `07-narrow-layout.tex`. These tested widths do not certify every formulation or class.

10 Engine compatibility

The example and regression suite has been executed under pdfLaTeX, XeLaTeX, and LuaLaTeX on Windows/MiKTeX. Unicode-engine smoke tests use fontspec and unicode-math with Latin Modern fonts. Inherited fonts and accented prose are covered; all fonts, publisher classes, and operating systems are not certified. A LaTeX kernel dated 2022-06-01 or newer is required; amsmath is the only external package dependency of the toolkit. The guide additionally uses geometry, array, longtable, fontenc, and hyperref.

11 Limitations and development status

Version 0.1.0 is experimental. ORLaTeX does not parse arbitrary mathematics, infer set membership, solve models, or validate their mathematical meaning. Explicit breaks are needed for some long expressions. Nested ORLaTeX environments, verbatim in mathematical content, and macro-generated row structure are unsupported. Models align expression starts rather than arbitrary relations. Very narrow contexts can overflow and are diagnosed.

An internally multiline row cannot split across pages. Continuation titles and subject-to markers are not repeated on later pages. Measurement can evaluate material more than once: avoid global side effects, counter increments, and labels hidden inside mathematical groups. Use top-level lightweight labels or structured label keys. Braced groups are opaque to shorthand normalization; optional/unbraced macro arguments cannot be inferred, so use native operators or a protective group for literal content.

11.1 Copyright and license

Copyright 2026 Imanol Felix Supo Mamani. Licensed under the LaTeX Project Public License, version 1.3c. LPPL maintenance status: author-maintained. Current Maintainer: Imanol Felix Supo Mamani. See LICENSE, NOTICE.md, and MANIFEST.txt for the grant and work's scope. No public release has occurred.

11.2 Reproduce the checks

From the repository root:

```
l3build check
python scripts/compile.py --engine all
l3build doc
python scripts/check_manual.py --engine all
```

The last command extracts and compiles the executable guide examples. Build commands above are shell commands, not LaTeX source. Actual results and verification limits are recorded in TEST-RESULTS.md.